

13-数据库

刘亚雄

极客时间-Java 讲师



三、MySQL事务篇

3.1 事务回顾

01-什么是事务？

- 事务指的是逻辑上的一组操作，这组操作的组成单元**要么全都成功，要么全都失败**
- 本质上是个**并发编程问题**
- Client与MySQL建立连接，一个连接开启一次会话Session，每个会话中可以多次开启和提交事务
- 事务由存储引擎实现，MySQL中只有InnoDB支持事务

02-事务四大特性ACID

- 原子性**Atomicity**：事务最小工作单元，要么全成功，要么全失败
- 一致性**Consistency**：事务前后数据的完整性必须保持一致
- 隔离性**Isolation**：多个用户并发使用数据库时，彼此事务操作数据不能互相干扰，所以要隔离。**隔离性与隔离级别有什么关系？**
- 持久性**Durability**：事务一旦提交，它对数据的改变就是永久性的

3.1 事务回顾

03-事务并发问题

- ① 脏读：一个事务读到了另一个事务**未提交**的数据
- ② 不可重复读：一个事务读到另一个事务**已经Update**的数据。引发事务中的多次查询结果不一致
- ③ 虚读/幻读：一个事务读到另一个事务**已经Insert**的数据。导致事务中多次查询的结果不一致

04-事务隔离级别

- **读未提交RU**：Read Uncommitted，一个事务读到另一个事务没有提交的数据
 - 存在：3个问题：脏读、不可重复读、幻读
- **读已提交RC**：Read Committed，一个事务读到另一个事务已经提交的数据
 - 存在：2个问题：不可重复读、幻读
- **可重复读RR**：Repeatable Read，在一个事务中读到的数据始终保持一致，无论另一个事务是否提交
 - 存在：1个问题：幻读
- **串行化读**：Serializable，同时只能执行一个事务，相当于事务中的单线程。所以啥问题也没有了

性能：串行化读 < 可重复读RR < 读已提交RC < 读未提交RU

安全性：串行化读 > 可重复读RR > 读已提交RC > 读未提交RU

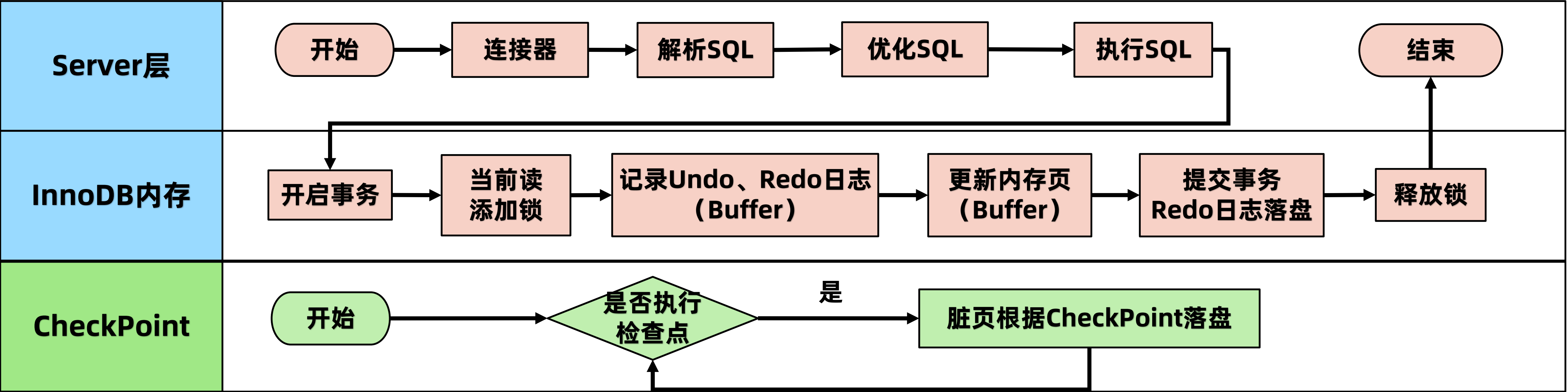


一条插入语句如何执行？与事务有什么关系？

3.2 一条Insert语句的执行流程

插入一条数据

```
1 | INSERT INTO t VALUES(1, '刘备');
```



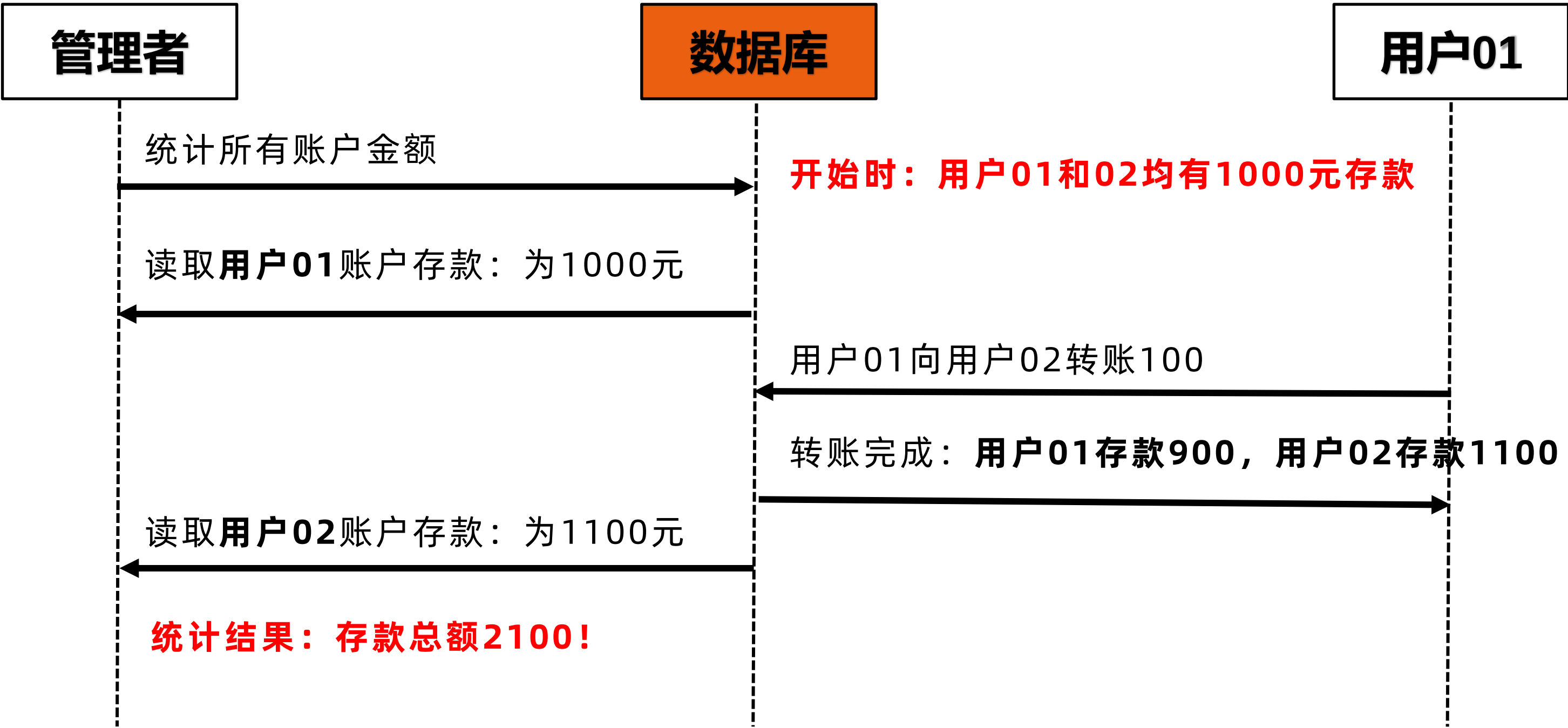
3.3 事务底层原理

01-丢失更新问题

两个Session针对同一数据进行修改操作时会存在丢失更新，这个现象称之为丢失更新问题



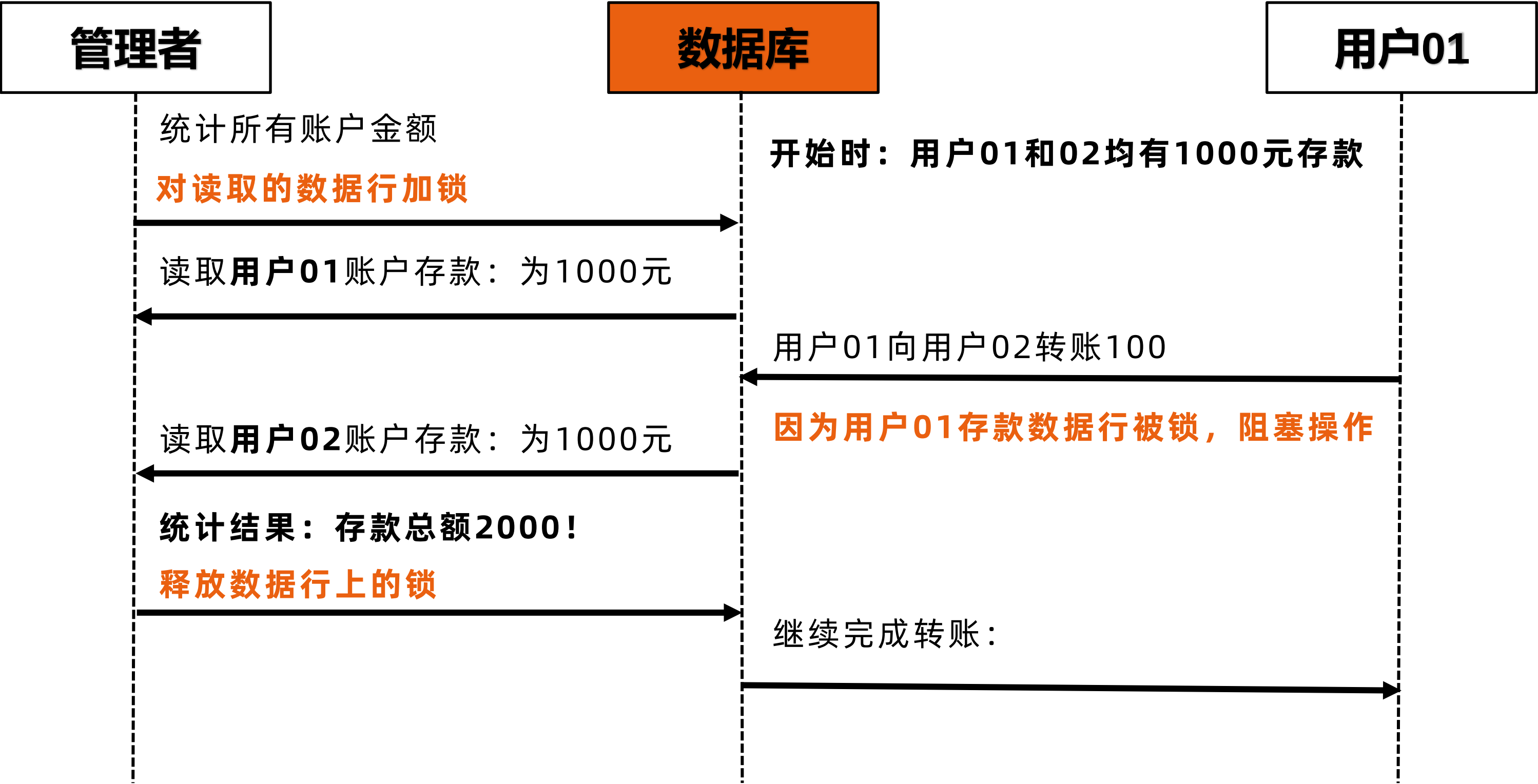
转账案例：



3.3 事务底层原理

02-如何解决丢失更新问题？

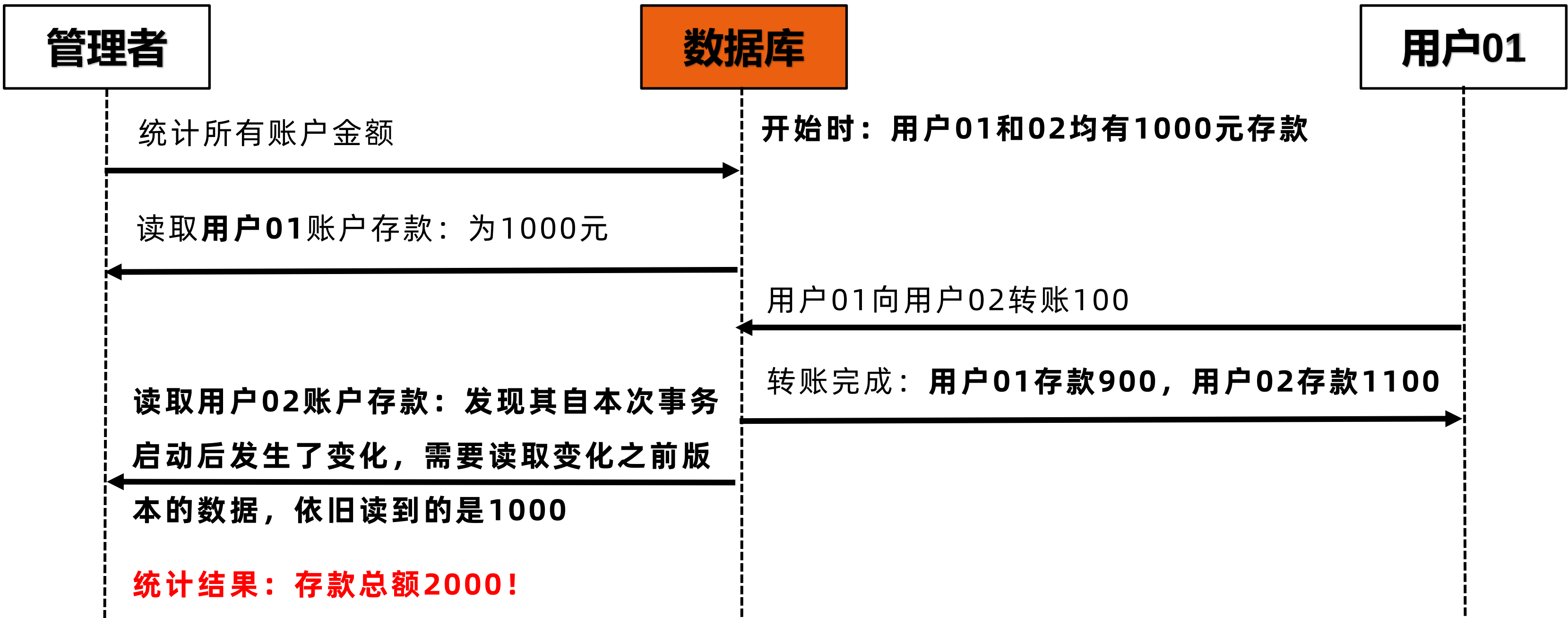
- 解决方案一：基于锁并发控制**LBCC**：查询总额的事务对读取数据行加锁，等读取数据结束再释放锁
- 解决方案二：基于版本并发控制**MVCC**



3.3 事务底层原理

02-如何解决丢失更新问题？

- 解决方案一：基于锁并发控制**LBCC**：查询总额的事务对读取数据行加锁，等读取数据结束再释放锁
- 解决方案二：基于版本并发控制**MVCC**：查询总额的事务不加锁，但是会读取事务开始时的快照数据



读不加锁，读写不冲突，MVCC保障了隔离性，如何实现的呢？

3.3 事务底层原理

03-什么是MVCC? 事务MVCC实现原理

- MVCC全称叫多版本并发控制，是RDBMS常用的一种并发控制方法，用来对数据库数据进行并发访问，实现事务。**核心思想：读不加锁，读写不冲突。**
- 实现原理：**数据快照**，不同事务访问数据快照中不同版本的数据
- 关键要素：Undo日志和ReadView

事务ID 回滚指针			表可见字段			
表默认字段	TRX_ID	Roll_Pointer:ID	name	age	address	
RowID-10	10	NULL	10	麦麦	23	beijing
RowID-10	20	aaaaa	10	雄雄	18	beijing
RowID-10	30	bbbbbb	10	迪迪	16	beijing

← 最新一条

← 同一行不同版本

Undo日志

m_low_limit_id	m_ids	m_up_limit_id	m_creator_trx_id
10	15、 20	21	10
10	10、 20	21	15
10	10、 15	21	20

ReadView

3.3 事务底层原理-Undo日志

04-Undo日志

- 在对数据修改时，InnoDB会产生一定量的Undo日志，如果事务执行失败Rollback了，则利用Undo日志回滚到历史版本
- Undo日志还可以解决**丢失更新问题**
- 存储位置：系统表空间内的回滚段中



看一下Undo日志究竟是怎么产生的！

事务ID 回滚指针 表可见字段			表默认字段 TRX_ID, Roll_Pointer:ID name age address			
RowID-10	1	NULL	10	麦麦	23	beijing
RowID-10	2	aaaaa	10	雄雄	18	beijing
RowID-10	3	bbbbbb	10	迪迪	16	beijing

Undo日志

05-Undo日志分类

- Insert Undo日志：是在Insert操作中产生的Undo日志
- Update Undo日志：是Update或Delete 操作中产生的Undo日志

3.3 事务底层原理-Undo日志

06-案例：Undo日志如何产生及回收

Insert into user(id,name,age,address) values (10,'麦麦',23,'beijing')

事务01：初始状态，Insert操作记录只对本事务可见，对其他事务不可见，所以事务提交后直接删除Undo日志无需回收

TRX_ID		Roll_Pointer				
RowID	1	NULL	10	麦麦	23	beijing

3.3 事务底层原理-Undo日志

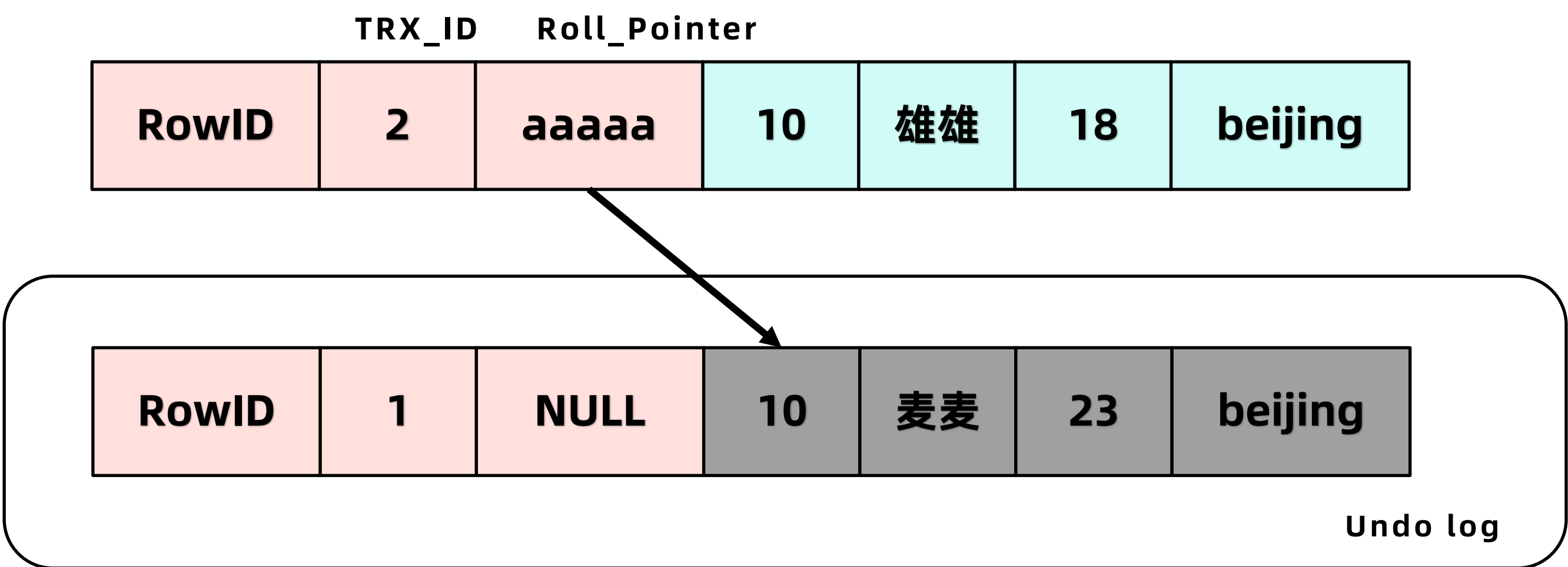
06-案例：Undo日志如何产生及回收

Insert into user(id,name,age,address) values (10,'麦麦',23,'beijing')

事务01：初始状态，Insert操作记录只对本事务可见，对其他事务不可见，所以事务提交后直接删除Undo日志无需回收

update user set name='雄雄',age=18 where id=10;

事务02：第一次修改，Update操作对已经存在行记录产生影响，为了实现MVCC，修改提交事务后，不能立即删除Update Undo日志而是会将其存入Undo日志链表中，等待Purge线程回收。



3.3 事务底层原理-Undo日志

06-案例：Undo日志如何产生及回收

Insert into user(id,name,age,address) values (10,'麦麦',23,'beijing')

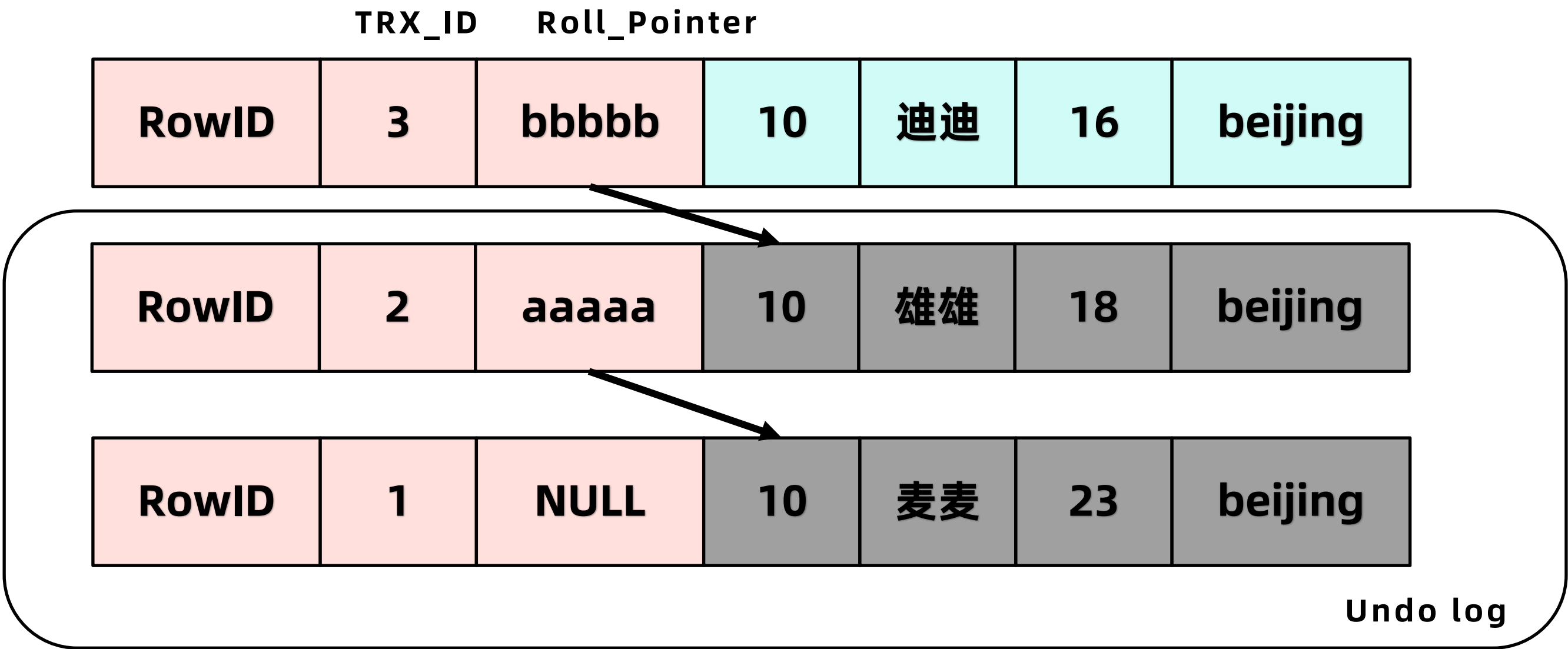
事务01：初始状态，Insert操作记录只对本事务可见，对其他事务不可见，所以事务提交后直接删除Undo日志无需回收

update user set name='雄雄',age=18 where id=10;

事务02：第一次修改，Update操作对已经存在行记录产生影响，为了实现MVCC，修改提交事务后，不能立即删除Update Undo日志而是会将其存入UndoLog链表中，等待Purge线程回收。

update user set name='迪迪',age=16 where id=10;

事务03：第二次修改



如何判断那个版本数据对当前事务可见？

3.3 事务底层原理-ReadView

07-什么是ReadView?

ReadView是张存储事务id的列表，主要包含当前系统中有哪些活跃的读写事务id，结合Undo日志的默认字段【trx_id】来控制那个版本的Undo日志可被其他事务看见。

四个列：

- **m_ids**：生成ReadView时，当前**活跃的事务id列表**
- **m_low_limit_id**：**事务id下限**，当前活跃事务中最小的事务id
- **m_up_limit_id**：**事务id上限**，生成ReadView时，应该分配给下一个事务的id值
- **m_creator_trx_id**：生成该ReadView的事务的事务id

m_low_limit_id	m_ids	m_up_limit_id	m_creator_trx_id
10	15、 20	21	10
10	10、 20	21	15
10	10、 15	21	20

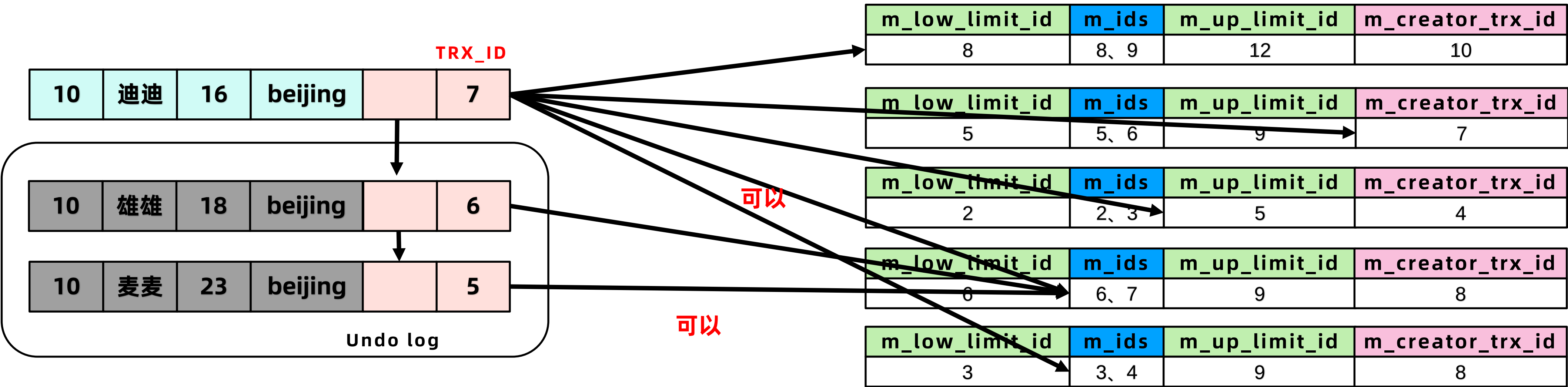
08-什么时候生成ReadView?

- 开启事务之后，在第一次查询(select)时，生成ReadView
- RC和RR隔离级别的差异本质是因为ReadView的生成时机不同

3.3 事务底层原理-ReadView

09-如何判断可见性？

- 开启事务执行第一次查询时，首先生成ReadView，然后依据Undo日志和ReadView按照判断可见性
- 判断规则如下：
 - ① TRX_ID 小于 m_low_limit_id，可以被当前事务访问
 - ② TRX_ID 等于 m_create_trx_id，可以被当前事务访问
 - ③ TRX_ID 大于 m_up_limit_id，不可以被当前事务访问
 - ④ TRX_ID 在low和up的limit之间，则需要判断是否在m_ids里面
 - ⑤ 如果在，则不可以访问，如果不在，则可以访问



3.4 事务ReadView案例

案例-RC与RR隔离级别下的可见性分析

每次读取数据前都生成一个ReadView，默认tab_user表中只有一条数据

```
1 CREATE TABLE `tab_user` (  
2   `id` int(11) NOT NULL,  
3   `name` varchar(100) DEFAULT NULL,  
4   `age` int(11) NOT NULL,  
5   `address` varchar(255) DEFAULT NULL,  
6   PRIMARY KEY (`id`)  
7 ) ENGINE=InnoDB;  
8 Insert into tab_user(id,name,age,address) values (1,'刘备',18,'蜀国');
```

RC：读已提交

- T6时刻：name的值是？
- T8时刻：name的值是？
- T10时刻：name的值是？

RR：可重复读

- T6时刻：name的值是？
- T8时刻：name的值是？
- T10时刻：name的值是？

时间	事务01【db_trx_id=100】	事务02【db_trx_id=200】	事务03【db_trx_id=300】
T1	开启事务	开启事务	开启事务
T2	更新为关羽	...	...
T3	更新为张飞	...	...
T4		更新为赵云	
T5		更新为诸葛亮	
T6			SELECT01, id=1, name为?
T7	提交事务01		
T8			SELECT02, id=1, name为?
T9		提交事务02	
T10			SELECT03, id=1, name为?

3.4 事务ReadView案例

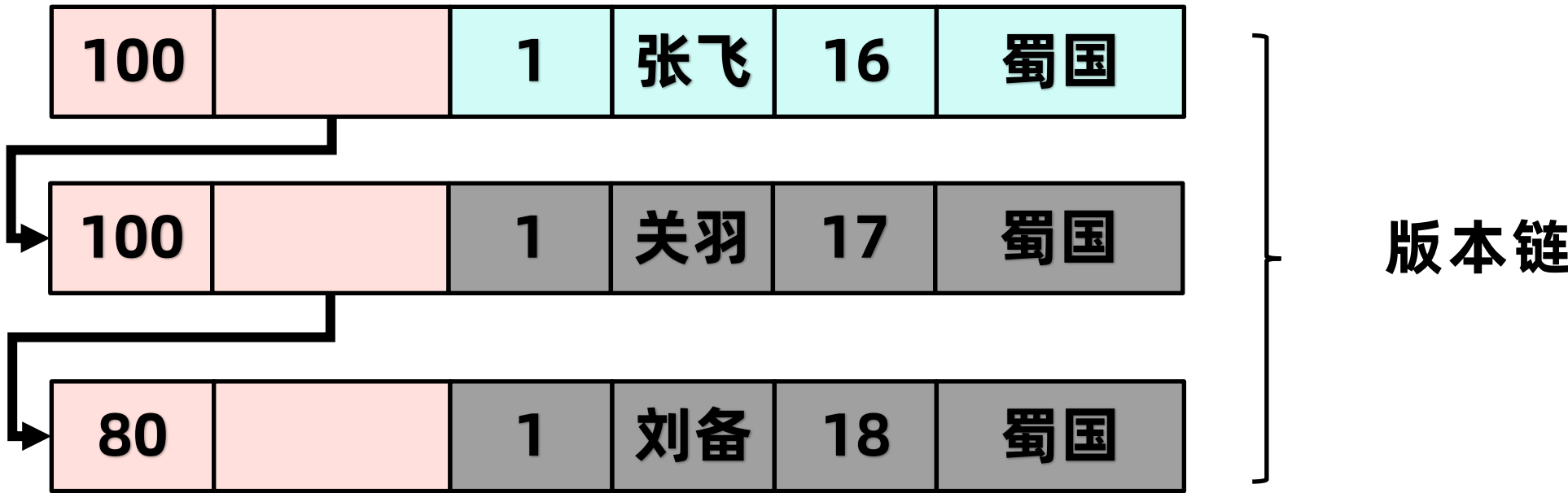
案例-RC与RR隔离级别下的可见性分析

每次读取数据前都生成一个ReadView，默认t表中只有一条数据

T1时刻：一条刘备的记录

80		1	刘备	18	蜀国
----	--	---	----	----	----

T3时刻：版本链如下



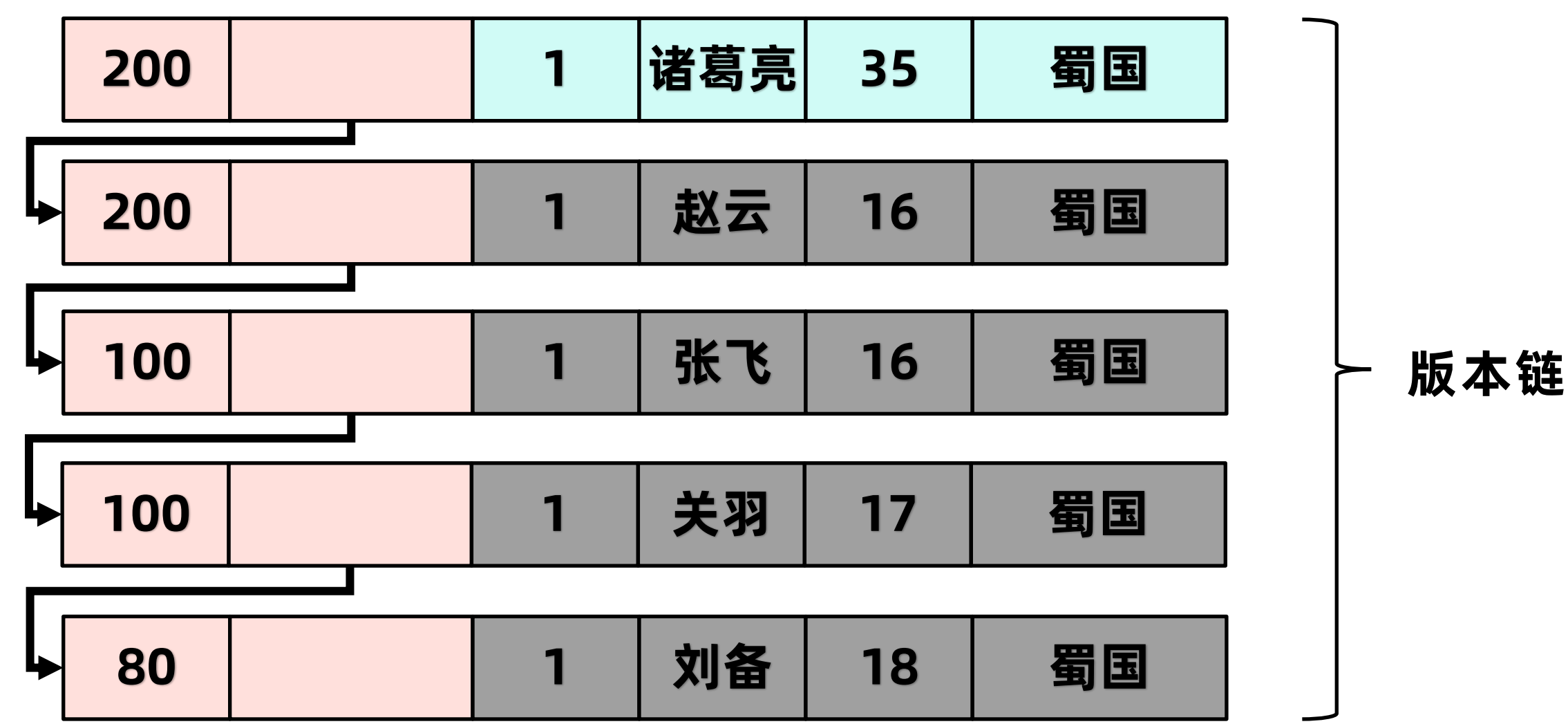
时间	事务01【db_trx_id=100】	事务02【db_trx_id=200】	事务03【db_trx_id=300】
T1	开启事务	开启事务	开启事务
T2	更新为关羽	...	...
T3	更新为张飞	...	...
T4		更新为赵云	
T5		更新为诸葛亮	
T6			SELECT01, id=1, name为?
T7	提交事务01		
T8			SELECT02, id=1, name为?
T9		提交事务02	
T10			SELECT03, id=1, name为?

3.4 事务ReadView案例

案例-RC与RR隔离级别下的可见性分析

每次读取数据前都生成一个ReadView，默认t表中只有一条数据

T5时刻：版本链如下



时间	事务01【db_trx_id=100】	事务02【db_trx_id=200】	事务03【db_trx_id=300】
T1	开启事务	开启事务	开启事务
T2	更新为关羽	...	...
T3	更新为张飞	...	...
T4		更新为赵云	
T5		更新为诸葛亮	
T6			SELECT01, id=1, name为?
T7	提交事务01		
T8			SELECT02, id=1, name为?
T9		提交事务02	
T10			SELECT03, id=1, name为?

 RU和Serializable隔离级别需要MVCC吗?

3.5 事务下的读操作：快照读&当前读

在MVCC中，读可以分为两类：**快照读Snapshot Read**与**当前读Current Read**

01-快照读

➤ 读不加锁，读取的是**版本链**的快照数据，默认的读都是快照读

02-当前读

➤ 读加锁，读取的永远都是最新数据，保证其他事务不会再并发修改这条记录



快照读

```
1 select * from table where ?;
```

不加锁

```
3 select * from table where ? lock in share mode;
```

加读锁（S锁）

```
5 select * from table where ? for update;
```

加写锁（X锁）

当前读

```
7 insert into table values (...);
```

加写锁（X锁）

```
9 update table set ? where ?;
```

保证其他并发事务不能修改

```
11 delete from table where ?;
```

3.5 事务下的读操作：快照读&当前读

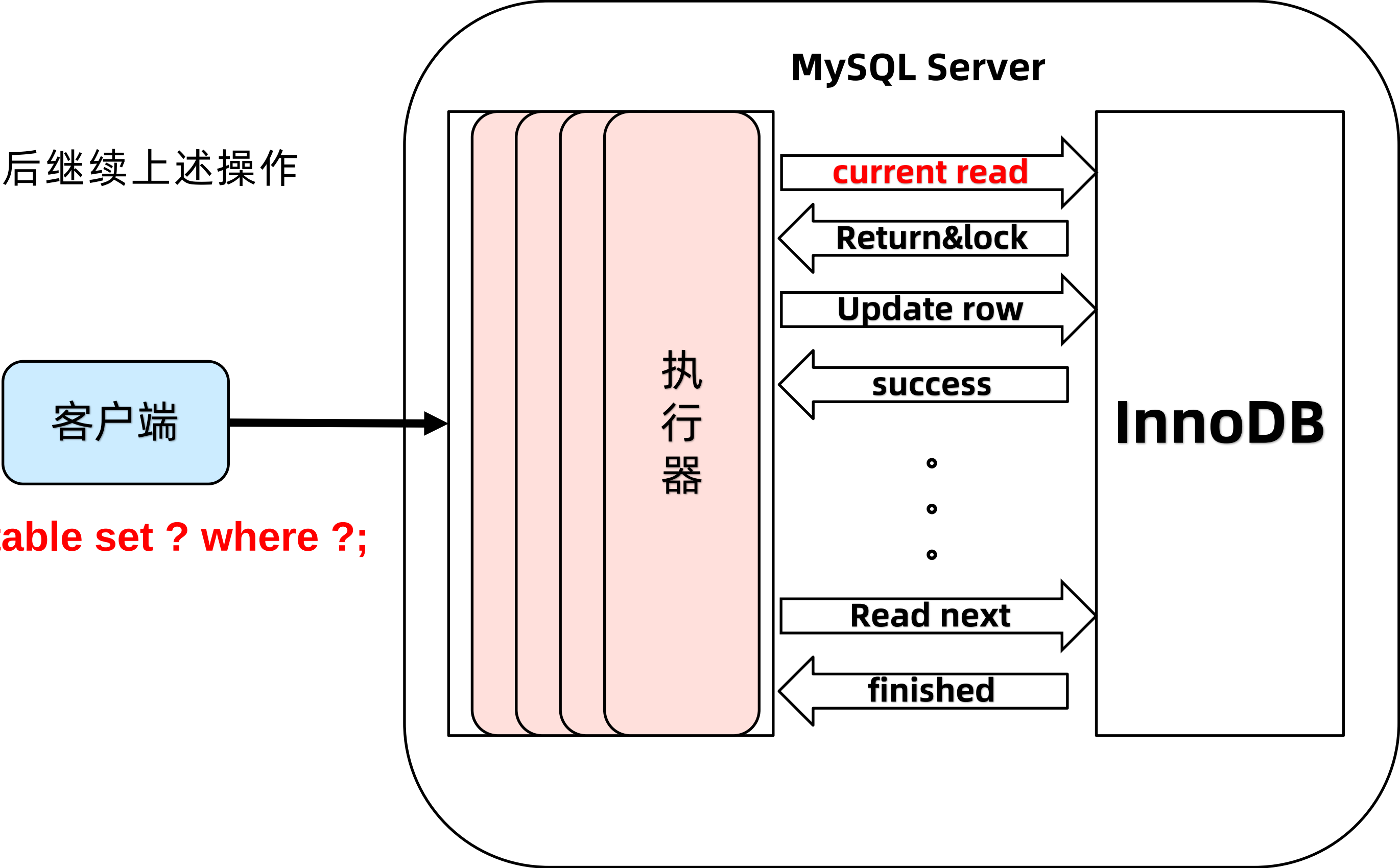
03-CRUD的新增、修改、删除操作的具体流程

当Update SQL操作进入MySQL后：

- ① MySQL Server根据Where条件，先从InnoDB读回第一条符合条件记录，**当前读**
- ② MySQL Server收到记录，并对记录加写锁
- ③ 加锁后发起Update操作更新记录
- ④ 本条记录操作完毕后，接着读取下一条，然后继续上述操作
- ⑤ 直到没有满足条件的记录为止
- ⑥ 提交事务，释放锁

Delete的操作与Update一样
Insert的操作与Update也差不多
区别：Insert操作会有唯一性约束等规则校验

update table set ? where ?;



THANKS

 极客时间 | 训练营

教育不是注满一桶水，而是点燃一把火